

# Vulkan Project Verification & Q&A

---

## 1. Confirmations

### Shading Model

#### Status: COMPLIANT

- **Observation:** The shading model implemented in `shaders/default.frag` uses the **Cook-Torrance** specular BRDF with a **Beckmann** distribution function (`D_Beckmann`) and Schlick approximations for Fresnel. The diffuse component is standard Lambertian.
- **Verification:** The code explicitly avoids GGX/Trowbridge-Reitz distributions, adhering to the requirement "Implementing a different shading model, including GGX-based ones, will not be accepted." You are using the compliant localized shading model.

### Vulkan Feature Set

#### Status: COMPLIANT

- **Synchronization2: Verified.** The project uses `VkCmdPipelineBarrier2` (via `labut2::image_barrier` in `synch.cpp`), which is the modern synchronization standard.
- **Dynamic Rendering: Verified.** Render passes and framebuffers are absent. The renderer uses `VkCmdBeginRendering` with `VkRenderingInfo` (in `rendering.cpp`), confirming adherence to dynamic rendering.
- **No VkShaderModule: Verified.** The `setup.cpp` file creates pipelines by passing `VkShaderModuleCreateInfo` structures directly into the `pNext` chain of `VkPipelineShaderStageCreateInfo`. It does **not** create persistent `VkShaderModule` handles, satisfying the "without VkShaderModule" requirement.
- **Validation: Verified.** Debug utils are enabled in `vulkan_window.cpp`, and the codebase is structured to be validation-clean (though runtime verification is needed to guarantee 0 errors, the setup is correct).

---

## 2. Explanations

### 1. How does the mipmapping work?

#### Mechanism: Offline/Load-time.

- The project does not generate mipmaps at runtime (e.g., using `VkCmdBlitImage`).
- Instead, `lut::load_image_texture2d` (in `main.cpp`) loads textures that presumably already contain mip levels or uses a library (like `stb_image/ktx`) that handles it, or simply loads the base level.
- **Critical Detail:** For the debug visualization (Key 1), you created a specific `debugSampler` in `setup.cpp` with `anisotropyEnable = VK_FALSE`. This allows the `debug_mip.frag` shader to visualize the exact mip level being sampled without the smoothing effects of anisotropic filtering, making the levels appear as distinct blocks/colors.

### 2. Debug Modes (Keys 1-4) & Fullscreen Interaction

## How they work:

- **Input:** `camera.cpp` handles keys 1-4 and updates `state.renderMode`.
- **Selection:** Inside `main.cpp` loop, `state.renderMode` selects which pipeline to bind for the **Scene Pass** (Opaque draw):
  1. **Key 1 (Default):** Reverts to `pipe`. Shader: `default.frag` (Textured PBR). **Note:** This mode also automatically switches to `alphaPipe` (`alpha.frag`) for materials with alpha masks (like leaves), as handled in the draw loop.
  2. **Key 2 (Mipmap):** Binds `mipPipe`. Shader: `debug_mip.frag`. Shows texture LOD colors.
  3. **Key 3 (Depth):** Binds `depthPipe`. Shader: `debug_depth.frag`. Shows non-linear depth.
  4. **Key 4 (Derivatives):** Binds `derivPipe`. Shader: `debug_deriv.frag`. Shows `fwidth` of depth.
- **Fullscreen Interaction:**
  - The debug modes affect what is drawn into the **Offscreen Buffer**.
  - The **Fullscreen Pass** (Post-Process) simply takes this Offscreen Buffer (as a texture) and draws it to the screen.
  - **Result:** It does **not** interfere; it faithfully displays whatever debug visualization was rendered to the offscreen buffer. The fullscreen pass applies Tone Mapping/Gamma on top of the debug colors, but the visualization remains valid.

## 3. Q&A

### 3. How many pipelines do we have?

**Total: 6**

1. **Triangle/PBR Pipeline** (`pipe`): Standard opaque rendering (Cook-Torrance).
2. **Alpha Pipeline** (`alphaPipe`): Same as PBR but handles alpha-masked materials (leaves).
3. **Mipmap Debug** (`mipPipe`): Visualization for Key 2.
4. **Depth Debug** (`depthPipe`): Visualization for Key 3.
5. **Derivatives Debug** (`derivPipe`): Visualization for Key 4.
6. **Post-Process** (`postProcPipe`): Fullscreen quad for tone mapping/gamma.

### 4. How does my PBR Shading work?

It calculates light interaction based on physics:

- **Diffuse:** Lambertian (Color / PI).
- **Specular:** Cook-Torrance BRDF using:
  - **D (Distribution):** Beckmann (how aligned the microfacets are).
  - **G (Geometry):** Cook-Torrance (self-shadowing of microfacets).
  - **F (Fresnel):** Schlick approximation (reflection intensity at angles).
- **Energy Conservation:** It handles metals vs dielectrics by conducting Fresnel conservation (multiplying diffuse by `1 - metalness`).

### 5. How does my alpha masking work?

- **Shader-based.**
- The `alpha.frag` shader samples the base color texture.

- **Logic:** `if( color.a < 0.5 ) discard;`
- **Pipeline Implementation:** It works by `vkCmdBindPipeline`. The system **automatically switches** not by magic, but because the CPU recording loop (`rendering.cpp`) checks every mesh's material: `if( material.alphaMaskTextureId != 0xffffffff ) ... use alphaPipe else use pipe`.
- If the alpha value is below threshold, the pixel is discarded entirely. It does **not** use alpha *blending* (transparency); it is a hard cutout (masking).

6. The fullscreen used to work even before we made a shader file for it. What are we using currently?

- **Current:** You are using `shaders/fullscreen.vert` and `shaders/fullscreen.frag`.
- **How it works without vertex buffer:** The vertex shader uses the "Big Triangle" trick. It generates vertices based on `gl_VertexIndex`:
  - Index 0: `(-1, -1)`
  - Index 1: `(3, -1)`
  - Index 2: `(-1, 3)`
  - This creates a triangle that covers the entire screen (and more), clipping correctly to the viewport. No vertex buffer inputs are required.

7. Explain swapchains and how we use them

- **What:** The swapchain is a collection of image buffers created by the Presentation engine (Window system) that we can draw into and show on screen.
- **Usage:**
  1. We acquire an image index (`vkAcquireNextImageKHR`).
  2. We transition it to `COLOR_ATTACHMENT_OPTIMAL`.
  3. We render our **Post-Process** pass into it.
  4. We transition it to `PRESENT_SRC_KHR`.
  5. We give it back to the presentation engine (`vkQueuePresentKHR`) to show on the monitor.

8. Are the keys 1 to 4 still gonna work in release mode?

Yes.

- The input handling in `camera.cpp` and the pipeline switching logic in `main.cpp` are **not** guarded by `#ifdef NDEBUG` macros. They are compiled into the release build and will function normally.

9. What do we do on the CPU and what on the GPU?

- **CPU:**
  - Handles Window/Input (GLFW).
  - Updates logic (Camera movement, `update_user_state`).
  - Updates Uniform Buffers (`update_scene_uniforms`).
  - **Records** commands into Command Buffers (doesn't execute them, just writes the "to-do list").
  - Submits work to the Queue.
- **GPU:**
  - Executes the command buffers.
  - Transforms vertices (Vertex Shader).
  - Rasterizes triangles.

- Calculates pixels (Fragment Shader).
- Writes to memory (Images).

## 10. Is the whole application using sRGB?

### Yes, in a corrected workflow:

1. **Inputs:** Textures are loaded as **sRGB** (`main.cpp` line 122). The GPU hardware converts them to **Linear** space when sampled in the shader.
2. **Processing:** All PBR lighting happens in **Linear** space (physically correct).
3. **Storage:** The Offscreen Buffer is **R16G16B16A16\_SFLOAT** (Linear).
4. **Output:** The Swapchain format is **sRGB** (selected in `vulkan_window.cpp`).
5. **Correction:** `fullscreen.frag` tone maps the linear color. It explicitly **removed** the manual `pow(color, 1/2.2)` because writing to an **sRGB** format image (the swapchain) allows the GPU to perform the Gamma correction automatically and efficiently at the end.

## 11. What presentation modes do we use and for what?

- **Logic:** `vulkan_window.cpp` prefers **VK\_PRESENT\_MODE\_FIFO\_RELAXED\_KHR**.
- **Fallback:** If relaxed is unavailable, it uses **VK\_PRESENT\_MODE\_FIFO\_KHR** (V-Sync).
- **Purpose:** **FIFO** prevents screen tearing by syncing with the monitor's refresh rate. **RELAXED** allows for immediate presentation if the frame is late, reducing stutter/lag at the cost of potential minor tearing.

## 12. Where is my light source?

- **Hardcoded.**
- In `default.frag` (and `alpha.frag`), line 27:

```
const vec3 LIGHT_POS = vec3(0.0, 15.0, 0.0);
```

- It is a static point light at **(0, 15, 0)** in World Space.